

Exporting Blender Character Animation for Robot Motion

A practical, easy-to-follow guide for converting mocap or keyframed character animation into safe robot joint data

The key idea

A robot usually does not need the character mesh or Blender control rig. It needs a timed sequence of values for its physical joints.

Animated Blender
character

Robot-matching
export rig

Baked joint motion

Filtered joint data

Robot controller

Character animation -> robot-specific motion -> safety checks -> hardware

Use this guide when you have

- Mocap or keyframed animation in Blender.
- A character rig that does not exactly match the robot.
- A robot controller that expects BVH, FBX, CSV, JSON, ROS 2, or proprietary joint commands.

Safety note

Always validate the motion in a simulator or digital robot model before sending it to hardware. Begin physical tests at low speed with an emergency stop available.

Prepared as a workflow reference. Adapt all limits, units, coordinate systems, and controller settings to the exact robot manufacturer documentation.

1. Collect the robot specifications

Before touching the Blender export settings, find out what the robot can physically do and what its software accepts. This prevents most retargeting and axis problems later.

Information to request

- [] Exact joint names and parent-child hierarchy.
- [] Rotation axis for each joint.
- [] Positive rotation direction for each axis.
- [] The robot home pose or zero pose.
- [] Minimum and maximum angle for each joint.
- [] Maximum joint speed and acceleration.
- [] Expected units: degrees or radians.
- [] Expected sample rate or update frequency.
- [] Accepted input format or software API.

Why this matters

Robot detail	What can go wrong without it
Joint axis	A shoulder or elbow may rotate in the wrong direction.
Zero pose	Every exported angle can have a constant offset.
Joint limits	The animation can request positions the robot cannot reach.
Speed limits	A visually smooth motion may still be too fast for the motors.
Units	Degrees interpreted as radians can create extreme commands.
Sample rate	Motion can become choppy, too slow, or too fast.

ROS terminology

In many ROS-based systems, the robot structure is described with URDF. The animation itself is commonly sent as a timed joint trajectory containing joint names and positions, with optional velocity and acceleration values.

Decide the final output before building the export rig

Final destination	Typical output
Mocap or retargeting application	BVH
Animation or DCC application	FBX
Custom robot software	CSV or JSON joint angles
ROS / ROS 2 controller	JointTrajectory or controller-specific messages
Manufacturer software	Proprietary SDK or motion format

2. Build a robot-matching export armature

Do not export the full animation control rig. Create a simplified duplicate or a separate armature that represents the robot motors.

Recommended setup

```

Character_Control_Rig
  | constraints / retargeting
  v
ROBOT_EXPORT
  | baked animation
  v
BVH / FBX / CSV / ROS trajectory

```

Rules for the export armature

- Use one controllable channel for each physical robot motor.
- Match robot joint names exactly whenever possible.
- Match the robot parent-child hierarchy.
- Align each bone local axis with the physical joint axis.
- Make the armature rest pose match the robot zero pose.
- Remove control shapes, IK helpers, twist helpers, and deformation-only bones that the robot does not use.
- Avoid animated scale.
- Export root translation only if the robot has a mobile base or another system that supports it.

A common mismatch

Character rig	Robot representation
upper_arm.L	left_shoulder_pitch

Character rig	Robot representation
One freely rotating shoulder bone	left_shoulder_roll
	left_shoulder_yaw
forearm.L	left_elbow_pitch
hand.L	left_wrist_yaw

Important

A Blender shoulder bone can rotate freely in three dimensions, while a robot shoulder may use two or three separate one-axis motors. The export rig must match the physical robot, not the original character skeleton.

Axis validation test

1. Put the robot export rig in its zero pose.
2. Rotate one export joint by a small positive value, such as +10 degrees.
3. Confirm the digital robot moves in the expected direction.
4. Repeat for every joint before processing a full animation.

3. Retarget the character animation

The ROBOT_EXPORT armature should follow the animated character, but remain restricted to movements the robot can reproduce.

Useful Blender constraints

Constraint	Typical use
Copy Rotation	Follow a source bone orientation.
Copy Location	Follow supported root or base translation.
Transformation	Remap one range or axis into another.
Limit Rotation	Prevent unsupported or unsafe angles.
IK	Solve a hand or foot target before converting to robot joints.

Restrict each joint to its real degrees of freedom

For a one-axis elbow motor, only one rotation channel should change:

```
left_elbow_pitch
  X Rotation: animated
  Y Rotation: locked
  Z Rotation: locked
```

Expect to modify the source animation

- Reduce extreme poses the robot cannot reach.
- Remove motions that require missing spine, wrist, finger, or shoulder joints.
- Adjust foot placement if the robot cannot reproduce the character stride.
- Keep the robot center of mass and balance limitations in mind.
- Watch for self-collisions and collisions with the floor or environment.

Best practice

Keep the original character rig untouched.
Perform retargeting, limits, cleanup, and baking
on a duplicate export scene or export rig.

Before baking

- [] Scrub the full timeline and verify every robot joint.
- [] Confirm there are no sudden 180-degree or 360-degree flips.
- [] Check that the export rig starts in a known pose.
- [] Set the scene frame rate to the intended animation timing.

4. Bake the final motion

Baking converts constraints and retargeting results into ordinary keyframes that can be exported or sampled reliably.

Blender Bake Action steps

1. Select the ROBOT_EXPORT armature.
2. Enter Pose Mode.
3. Select all export bones.
4. Choose Pose -> Animation -> Bake Action.
5. Enter the complete frame range for the animation.
6. Bake and then test the armature with constraints disabled.

Suggested Bake Action settings

Setting	Recommended value	Reason
Frame Step	1	Samples every scene frame.
Only Selected Bones	On	Avoids baking unwanted rig controls.
Visual Keying	On	Records the final evaluated motion.
Clear Constraints	Optional	Use only on a duplicate/export rig.
Bake Data	Pose	Bakes pose-bone transforms.
Clean Curves	Off initially	Prevents early loss of motion detail.

Verify the bake

After baking, temporarily disable or remove the constraints. The export armature should continue moving the same way. If it changes, the bake did not capture all evaluated motion.

Animation timing

Blender frame rate	Seconds per frame
24 fps	0.04167 s
30 fps	0.03333 s
60 fps	0.01667 s

When exporting to timestamped joint data, use the animation time rather than assuming every robot controller runs at the Blender frame rate.

5. Process the motion for robot safety

A clean Blender animation is not automatically safe robot motion. Human mocap contains movements, speeds, and small jitters that physical motors may not reproduce safely.

Check every joint for

[] Position or angle limit violations.

- [] Sudden angle wrapping or Euler flips.
- [] Excessive joint velocity.
- [] Excessive acceleration or abrupt direction changes.
- [] High-frequency mocap jitter.
- [] Self-collisions.
- [] Collisions with the environment.
- [] Balance and support problems.
- [] Impossible foot or hand contact positions.

Recommended cleanup sequence

1. Convert all joints into the robot coordinate convention.
2. Apply zero-pose offsets and joint sign corrections.
3. Clamp or redesign poses that exceed position limits.
4. Smooth small high-frequency jitter without erasing intentional movement.
5. Resample at the controller update rate.
6. Calculate and limit velocity and acceleration.
7. Run collision and balance checks in simulation.

Never send raw mocap directly to hardware

Tiny movements that look harmless on a character can cause motor chatter, unstable balance, overheating, or rapid direction changes. Filter and validate the trajectory first.

Keep original and processed data separate

Version	Purpose
Source animation	Original artistic or mocap performance.
Retargeted animation	Movement mapped to the robot skeleton.
Limited animation	Joint positions kept within physical limits.
Filtered trajectory	Smoothed and resampled controller input.
Hardware test version	Reduced speed and conservative range.

6. Choose and export the correct format

Option A - BVH for mocap interchange

Use BVH when the next application specifically expects a skeleton and motion-capture animation.

Blender: File -> Export -> Motion Capture (.bvh)

- Confirm the frame range and scale.
- Confirm root translation requirements.
- Check forward and up-axis conversion.
- Check rotation order.
- Treat BVH as an intermediate format unless the robot software explicitly accepts it.

Option B - FBX for animation applications

Use FBX when another DCC, engine, retargeter, or conversion tool needs the baked skeleton animation.

FBX setting	Suggested value
Selected Objects	On
Object Types	Armature; Mesh optional
Only Deform Bones	On only if all required robot bones are deform bones
Add Leaf Bones	Off
Bake Animation	On
Key All Bones	On
NLA Strips	Off for one animation
All Actions	Off for one animation
Sampling Rate	1
Simplify	0.0

FBX limitation

Blender constraints are not transferred as functioning constraints. Their evaluated motion must be baked into keyframes before export.

Option C - CSV or JSON joint-angle data

This is often the clearest final handoff for custom robot software because every row is an explicit timestamp and every column is a robot joint.

```
time,left_shoulder_pitch,left_shoulder_roll,left_elbow_pitch
0.000,0.000,0.000,0.150
0.033,0.021,-0.006,0.174
0.067,0.044,-0.011,0.201
```

- Use the robot-defined zero position.
- Use the expected units.
- Keep joint column order consistent.
- Include timestamps.
- Apply limits before saving the final file.

7. ROS 2 trajectory output

For ROS-based robots, the processed joint data is commonly converted into a joint trajectory or another controller-specific message.

Conceptual JointTrajectory structure

```
joint_names:
  left_shoulder_pitch
  left_shoulder_roll
  left_elbow_pitch

point 0:
  positions: [0.000, 0.000, 0.150]
  time_from_start: 0.000

point 1:
  positions: [0.021, -0.006, 0.174]
  time_from_start: 0.033
```

Typical unit conventions

Joint type	Position	Velocity
Revolute joint	Radians	Radians per second
Prismatic joint	Meters	Meters per second

Controller compatibility

A correct JointTrajectory message is not enough by itself. The joint names, ordering, timestamps, controller interface, and trajectory tolerances must match the specific robot controller.

Data conversion checklist

- [] Bone name -> robot joint name.
- [] Blender local rotation -> robot joint angle.
- [] Blender axis direction -> robot positive direction.
- [] Blender rest pose -> robot zero offset.
- [] Scene frame -> time_from_start.
- [] Degrees -> radians when required.
- [] Filtered angle curves -> controller trajectory points.

A practical custom exporter

For direct robot output, a Blender Python script can sample the baked export armature frame by frame, convert each joint to the robot coordinate convention, and write CSV, JSON, or trajectory data. The conversion should be reviewed against a known test pose before exporting a full performance.

8. Test before using the physical robot

Recommended test order

1. Reimport the BVH or FBX into a clean Blender file, or reload the exported joint data.
2. Compare the result with the original animation.
3. Test a neutral pose and a small positive movement on every joint.
4. Run the full motion on a digital robot model or simulator.
5. Check joint limits, collisions, balance, velocity, and acceleration.
6. Create a reduced-speed hardware test version.
7. Place the physical robot in a clear test area.
8. Keep the emergency stop accessible.
9. Increase speed only after the conservative test passes.

Common problems and fixes

Problem	Likely cause	Fix
Joint moves backward	Axis sign mismatch	Invert that joint or correct the export-axis mapping.
Pose has a constant offset	Zero pose mismatch	Apply the correct home-pose offset.
Joint spins suddenly	Euler wrapping or rotation order	Unwrap curves or use a stable conversion method.
Motion is jittery	Raw mocap noise	Filter and resample the joint curves.
Motion is too fast	Timing or unit mismatch	Verify timestamps, frame rate, and controller rate.
Missing animation in FBX	Constraints not baked	Bake the evaluated pose before export.
Extra bones appear	Control or leaf bones exported	Export selected robot bones and disable leaf bones.
Feet slide or body tips	Kinematic mismatch	Retarget with contact and balance constraints.

Hardware testing rule

The first successful simulator test is not permission to run at full speed. Use conservative joint ranges and reduced playback speed for the first physical tests.

One-page quick reference

The complete workflow

1. Get robot joint specifications
2. Build ROBOT_EXPORT armature
3. Match names, hierarchy, axes, and zero pose
4. Retarget character motion
5. Lock unsupported axes and apply limits
6. Bake the evaluated animation
7. Export or sample joint data
8. Convert units, directions, and offsets
9. Filter, resample, and enforce limits
10. Simulate
11. Test physical hardware slowly

Blender export checklist

- [] ROBOT_EXPORT contains only required robot joints.
- [] Rest pose matches the robot zero pose.
- [] Each joint rotates around the correct local axis.
- [] Animation is baked with Visual Keying.
- [] No unsupported scale animation.
- [] No unbaked constraints remain.
- [] Frame rate and frame range are correct.
- [] Exported file has been reimported and checked.

Robot data checklist

- [] Joint names and ordering match the controller.
- [] Units are correct.
- [] Joint signs and zero offsets are correct.
- [] Positions remain inside limits.
- [] Velocity and acceleration remain inside limits.
- [] Motion is filtered and resampled.
- [] Simulation passes without collision or balance issues.
- [] First hardware test uses reduced speed.

Useful reference documentation

Blender BVH import/export: https://docs.blender.org/manual/en/latest/addons/import_export/anim_bvh.html

Blender FBX import/export: https://docs.blender.org/manual/en/latest/addons/import_export/scene_fbx.html

ROS 2 JointTrajectoryPoint: https://docs.ros.org/en/rolling/p/trajectory_msgs/msg/JointTrajectoryPoint.html

ROS joint limits: https://docs.ros.org/en/jazzy/p/joint_limits/